

PART I. FINAL SYSTEM ARCHITECTURE

20 points

The diagram below is the corrected architecture resulting from the Capstone System Cohesion Check. The main correction was that scoring had been drawn as a single scene-level component, which did not match the implementation: a level spans several Unity scenes, so the score is now shown as two separate responsibilities — a per-scene counter and a level-wide running total — feeding one save operation. The second correction was the placement of the speech recognizer, which is an operating-system service outside the system boundary rather than a component the group builds.

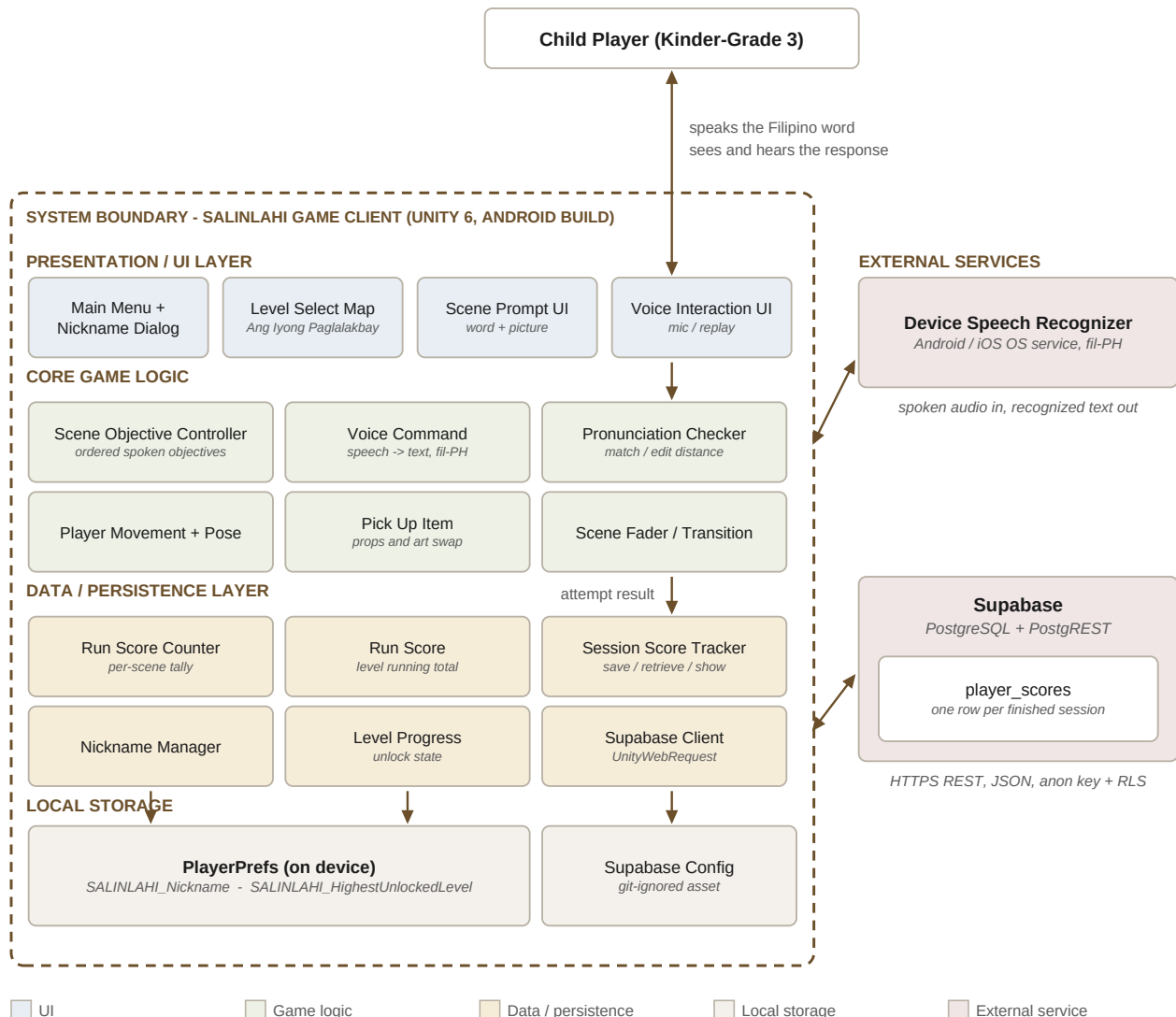


Figure 1. SALINLAHI system architecture. Dashed outline marks the system boundary; components outside it are services the group consumes but does not implement.

Architecture explanation

Overall structure. SALINLAHI is a single Unity client with a three-layer internal structure — presentation, core game logic, and a data/persistence layer — plus two external services. There is no custom backend server: the game talks directly to Supabase's REST interface, so the client is the only application the group deploys.

Where user interaction occurs. All interaction happens in the presentation layer. The child enters a nickname at the main menu, chooses an unlocked environment on the level-select map, then plays a scene where the prompt UI shows the Filipino word as text and picture and the voice interaction UI shows mic and replay buttons. The child's input is speech, not keyboard or controller.

Where major processing occurs. Processing is in the core game logic layer, inside the client. Voice Command passes captured audio to the device recognizer and receives recognized text; Pronunciation Checker decides whether that text matches the expected word, using normalisation, a per-word list of accepted variants, and a length-scaled Levenshtein tolerance, because child speech and Tagalog recognition are both noisy. Scene Objective Controller owns the ordered list of words for the scene and raises the configured effect when one is answered — moving the character, removing a prop, swapping the character art, or fading into the next scene.

Where persistent data is stored. In two places, for two different reasons. Session results are stored remotely in Supabase's `player_scores` table, because the evaluation phase needs results collected across devices and testers. Player identity and map progress are stored locally in `PlayerPrefs`, because they must work instantly and offline every time the app opens.

How components communicate. Inside the client, communication is by direct C# method calls and C# events — Voice Command raises `OnAnswerChecked`, which Run Score Counter and Scene Objective Controller both subscribe to, so the counter never has to be wired into each objective. Run Score is a static class rather than a scene object, so the running total survives scene loads within a level. Across the boundary, Supabase Client uses `UnityWebRequest` over HTTPS with JSON bodies.

External services. Two. The device speech recognizer (accessed through the `UnitySpeechToText` plugin, configured to `fil-PH`) is an OS capability available on Android and iOS only; in the Unity Editor it is unavailable, so a keyboard path feeds the identical code. Supabase provides PostgreSQL with a PostgREST API; the game authenticates with the anon public key and is restricted by Row Level Security to inserting and selecting on one table.